ADAMAS UNIVERSITY

KOLKATA

DATABASE MANAGEMENT SYSTEM

DBMS ASSIGNMENT

COURSE CODE: CSE303

Relational Calculus • Dependency Preservation • Query Optimization
Database Security • Access Control • SQL Injection
Data Warehousing • Data Mining • Intrusion Detection

SUBMITTED BY	SUBMITTED TO
Riya Sadhukhan	Abhinandan Ghosh
PROGRAMME	**COURSE**
BCA (Hons.)	CSE303 – Database Management System
SEMESTER / YEAR	**SUBMISSION DATE**
Semester V • Year 3rd	06 November 2026
ROLL NUMBER	**REGISTRATION NUMBER**
UG/SOET/29/24/075	AU/2024/000208

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

DECLARATION

This assignment has been prepared for the CSE303 Database Management System course. It presents core DBMS concepts with definitions, key features, practical examples, and illustrative SQL or pseudocode where appropriate.

I have made an effort to explain each topic in clear academic language and to distinguish related concepts that are often confused. SQL examples use broadly standard SQL; small syntax differences may exist between database products such as MySQL, PostgreSQL, Oracle, and SQL Server.

HOW TO READ THIS ASSIGNMENT

- **Definition:** the meaning and purpose of the concept.
- **Important features:** the main properties, rules, advantages, and limitations.
- **Real-world example:** how the concept is used in an organisation or application.
- **Code and output:** runnable or representative SQL examples wherever code makes the topic clearer.

Terminology note: the supplied topic list appears to contain a few typing errors. This assignment uses the standard DBMS terms relational calculus, dependency preservation, and query optimisation. The phrase "data working vs data making" is interpreted as data warehousing vs. data mining, matching the reference topic list. "Insertion and deletion" is covered both as SQL operations and as insertion/deletion anomalies in relational design.

TABLE OF CONTENTS

1. Relational Calculus: Tuple and Domain Relational Calculus
2. Dependency Preservation
3. Query Optimisation Algorithms
4. Database Security
5. DAC, MAC and RBAC Access-Control Models
6. Insertion and Deletion: SQL Operations and Anomalies
7. SQL Injection
8. Data Warehousing vs. Data Mining
9. Intrusion Detection in Databases
10. Summary and References

1. RELATIONAL CALCULUS

Relational calculus is a non-procedural, declarative query language for relational databases. It describes *what* data is required rather than the sequence of operations used to retrieve it. It is based on predicate logic and forms part of the theoretical foundation of relational query languages.

A relational-calculus expression specifies conditions that the required tuples or attribute values must satisfy. A query is evaluated over relations (tables), and the result is another relation. Two principal forms are Tuple Relational Calculus (TRC) and Domain Relational Calculus (DRC).

A. Tuple Relational Calculus (TRC)

Definition: TRC uses tuple variables that represent complete rows of a relation. A query has the general form **{ t | P(t) }**, meaning "the set of tuples t for which predicate P(t) is true."

Important features:

- Variables range over tuples (rows) of a relation.
- Conditions use comparisons, logical operators (AND, OR, NOT), and quantifiers such as ∃ (there exists) and ∀ (for all).
- It is declarative: it specifies the required result, not the execution plan.
- Safe expressions produce finite results drawn from the database domain; unrestricted calculus expressions can describe infinite sets.

Real-world example: a university wants the details of students enrolled in the BCA programme. The tuple variable represents each Student row, and the predicate checks the programme attribute.

```
{ s | Student(s) AND s.programme = 'BCA' }
```

Equivalent SQL:

```
SELECT *
FROM Student
WHERE programme = 'BCA';
```

```
OUTPUT
Returns every Student row whose programme is BCA.
```

B. Domain Relational Calculus (DRC)

Definition: DRC uses domain variables, where each variable represents an individual attribute value rather than an entire tuple. Its general form is **{ <x1, x2, …, xn> | P(x1, x2, …, xn) }**.

Important features:

- Variables range over individual attribute values (domains).
- The query explicitly identifies the values to be returned.
- Predicates connect variables to relation attributes and to one another.
- Like TRC, it is declarative and uses logical connectives and quantifiers.

Real-world example: retrieve the names of students whose programme is BCA. Assume Student(student_id, student_name, programme).

```
{ <name> | EXISTS id, prog
  (Student(id, name, prog) AND prog = 'BCA') }
```

Equivalent SQL:

```
SELECT student_name
FROM Student
```

```
WHERE programme = 'BCA';
```

```
OUTPUT
student_name
------------
Aarav
Diya
(Only matching rows are returned.)
```

TRC vs. DRC: TRC returns tuple variables (often whole rows); DRC returns selected domain values (often particular columns). Both have equivalent expressive power for safe relational-calculus queries and are closely related to relational algebra and SQL.

2. DEPENDENCY PRESERVATION

Dependency preservation is a desirable property of database decomposition during normalisation. It ensures that all functional dependencies of the original relation can be checked by examining the decomposed relations individually, without joining them back together.

Definition: Let a relation schema R be decomposed into R1, R2, …, Rn and let F be the set of functional dependencies. The decomposition is dependency-preserving if the union of the projected dependencies on the component schemas is equivalent to F (that is, it implies all dependencies in F).

Important features:

- Makes integrity constraints easier and less expensive to enforce.
- Allows each dependency to be checked within one decomposed table, where possible.
- Reduces the need for costly joins solely to validate constraints.
- Lossless-join decomposition and dependency preservation are different properties; a decomposition should ideally satisfy both.
- A decomposition may be lossless but fail to preserve every dependency.

Real-world example: a college maintains Student(student_id, student_name, dept_id, dept_name), with dependencies student_id → student_name, dept_id and dept_id → dept_name. Splitting department details into a separate Department table avoids repeating the department name for every student.

Before decomposition: Student(student_id, student_name, dept_id, dept_name).

After decomposition: Student(student_id, student_name, dept_id) and Department(dept_id, dept_name).

SQL implementation:

```
CREATE TABLE Department (
    dept_id   INT PRIMARY KEY,
    dept_name VARCHAR(60) NOT NULL
);

CREATE TABLE Student (
    student_id   INT PRIMARY KEY,
    student_name VARCHAR(80) NOT NULL,
    dept_id      INT NOT NULL,
    FOREIGN KEY (dept_id) REFERENCES Department(dept_id)
);
```

How preservation works: student_id → student_name is enforced by Student's primary key. dept_id → dept_name is enforced by Department's primary key. The foreign key ensures that each student's dept_id refers to an existing department. No join is needed to check either functional dependency.

Why it matters: without dependency preservation, a database may need joins to check a rule, increasing complexity and creating opportunities for inconsistent updates. During schema design, test losslessness and dependency preservation separately.

3. QUERY OPTIMISATION ALGORITHMS

Query optimisation is the process of choosing an efficient execution plan for a SQL query. The DBMS aims to reduce response time and resource use, such as disk I/O, CPU work, memory consumption, and network transfer.

Definition: A query optimiser examines equivalent ways to execute a query and selects a plan with an estimated low cost. The optimiser uses table statistics, indexes, cardinality estimates, and cost models.

Important features and stages:

- **Parsing and validation:** check SQL syntax, table names, columns, permissions, and data types.
- **Logical rewriting:** simplify predicates, push filters closer to base tables, remove unnecessary columns, and transform equivalent expressions.
- **Plan generation:** consider access paths, join orders, and join algorithms.
- **Cost estimation:** estimate rows, I/O, CPU, memory, and sometimes network costs.
- **Plan selection:** choose the estimated cheapest plan and execute it.

Common optimisation algorithms and techniques

1. Heuristic / rule-based optimisation: applies rules that are generally beneficial, such as performing selections early, projecting only needed columns, and replacing Cartesian product plus selection with a join. It is fast, but fixed rules may not be best for every dataset.

2. Cost-based optimisation: estimates the cost of alternative plans using statistics such as table row counts, value distributions, and index selectivity. It is widely used in modern DBMSs.

3. Join-order optimisation: chooses the order in which tables are joined. Dynamic programming can explore many join orders for a manageable number of relations; for large queries, optimisers may use heuristics or search strategies to limit planning time.

4. Join algorithms: nested-loop join is useful when one input is small or the inner side is indexed; sort-merge join is useful when inputs are sorted or need ordering; hash join is often efficient for equality joins on large unsorted inputs.

5. Index selection and access paths: an index can avoid scanning every row, but maintaining too many indexes increases storage and slows inserts, updates, and deletes.

Real-world example: an e-commerce site searches orders for one customer during a date range. An index on (customer_id, order_date) may allow the DBMS to locate matching orders quickly rather than scanning the entire Orders table.

```
CREATE INDEX idx_orders_customer_date
ON Orders(customer_id, order_date);

SELECT order_id, order_date, total_amount
FROM Orders
WHERE customer_id = 105
  AND order_date >= DATE '2026-01-01';
```

Illustrative EXPLAIN output (format differs by DBMS):

```
Index range scan on Orders using idx_orders_customer_date
Estimated rows: 18
Predicate: customer_id = 105
          AND order_date >= 2026-01-01

OUTPUT
The plan uses the composite index to narrow the rows examined. Actual plan and performance depend on the DBMS, statist
```

Practical optimisation guidelines: select only required columns instead of SELECT *; filter early; use suitable indexes; avoid unnecessary functions on indexed columns in predicates; inspect EXPLAIN / EXPLAIN ANALYZE; keep statistics current; and measure performance with representative data.

4. DATABASE SECURITY

Database security protects stored information from unauthorised access, disclosure, modification, destruction, and service disruption. It combines technical controls, policies, monitoring, and operational practices.

Definition: Database security is the collection of measures used to protect database systems, database accounts, data, and the services that process data.

Core security objectives – CIA triad

- **Confidentiality:** only authorised people and applications can view information.
- **Integrity:** data remains accurate, complete, and protected from unauthorised changes.
- **Availability:** authorised users can access data and services when needed.

Important features and controls

- **Authentication:** verifies identity using passwords, multi-factor authentication, certificates, or other credentials.
- **Authorisation:** determines which tables, rows, columns, and operations a user may access.
- **Encryption:** protects data at rest and in transit; key management is essential.
- **Auditing and logging:** records logins, queries, privilege changes, and sensitive data access.
- **Backup and recovery:** supports restoration after accidental deletion, hardware failure, or attack.
- **Patch and configuration management:** removes known vulnerabilities and disables unnecessary services.
- **Data masking and minimisation:** limits exposure of personal or confidential data.
- **Network protection:** restricts database connectivity using firewalls, private networks, and secure channels.

Real-world example: a hospital database stores patient records. Doctors may view records for treatment, billing staff may access billing details, and administrators manage accounts. Encryption, role-based permissions, audit logs, and backups help protect the records.

Illustrative SQL permissions:

```
-- Give a reporting account read-only access
GRANT SELECT ON Patient TO report_user;

-- Remove that access when it is no longer needed
REVOKE SELECT ON Patient FROM report_user;
```

Good practice: follow least privilege, use unique accounts, avoid shared administrator credentials, rotate and protect secrets, test backups, review audit logs, and separate development/test data from production data.

5. DAC, MAC AND RBAC ACCESS-CONTROL MODELS

Access control defines who can perform which action on which resource. DAC, MAC, and RBAC are three important models; they differ in who grants permissions and how access decisions are made.

A. Discretionary Access Control (DAC)

Definition: DAC allows the owner of an object (such as a table or file) to grant or revoke access to other users.

- Permissions are commonly granted through object ownership or SQL GRANT and REVOKE statements.
- Flexible and convenient for collaborative environments.
- Permissions can spread when users are allowed to grant access to others.
- The owner or administrator must manage permissions carefully.

Real-world example: a lecturer who owns a course-results table grants a teaching assistant read access.

```
GRANT SELECT ON CourseResults TO teaching_assistant;
REVOKE SELECT ON CourseResults FROM teaching_assistant;
```

B. Mandatory Access Control (MAC)

Definition: MAC enforces centrally defined security policies based on security labels assigned to users (clearances) and data (classifications). Individual data owners cannot freely override the policy.

- Uses classifications such as Public, Confidential, Secret, and Top Secret.
- Access is determined by centrally administered rules and labels.
- Strong control for highly sensitive or regulated information.
- Less flexible and more complex to administer.

Real-world example: in a defence organisation, a user with “Confidential” clearance cannot read a “Secret” document merely because another employee wants to share it.

C. Role-Based Access Control (RBAC)

Definition: RBAC assigns permissions to roles and assigns users to those roles. A user's access follows the roles they hold.

- Roles represent job functions such as Doctor, Nurse, Cashier, or Database Administrator.
- Permissions are managed centrally through role definitions.
- Supports least privilege and separation of duties.
- Role hierarchies and constraints can model organisational structures.

Real-world example: in a university system, an Admissions Officer can update applicant records, a Lecturer can view class lists and enter marks, and a Student can view their own results.

```
CREATE ROLE lecturer;
GRANT SELECT ON Student TO lecturer;
GRANT UPDATE (marks) ON ExamResult TO lecturer;

GRANT lecturer TO user_riya;
```

Comparison

Model	Basis of access	Typical strength	Typical limitation
DAC	Object owner grants permissions	Flexible sharing	Permission propagation risk
MAC	Central policy + security labels	Strict confidentiality	Less flexible
RBAC	User's assigned organisational role	Manageable at scale	Role design can become complex

In practice, organisations may combine approaches: RBAC for everyday job permissions, MAC-like classifications for highly sensitive records, and DAC-style grants for controlled sharing.

6. INSERTION AND DELETION: SQL OPERATIONS AND DESIGN ANOMALIES

INSERT and DELETE are data-manipulation operations. Poorly designed tables can also suffer insertion and deletion anomalies, which normalisation helps reduce.

A. INSERT operation

Definition: INSERT adds one or more new rows to a table. Values must satisfy data types, NOT NULL, UNIQUE, CHECK, primary-key, and foreign-key constraints.

Example: add a new student to a university database.

```
INSERT INTO Student(student_id, student_name, programme)
VALUES (201, 'Riya', 'BCA');
```

```
OUTPUT
1 row inserted.
```

A multi-row insert can add several records in one statement (supported syntax varies by DBMS):

```
INSERT INTO Student(student_id, student_name, programme)
VALUES
  (202, 'Aarav', 'BCA'),
  (203, 'Diya', 'BBA');
```

```
OUTPUT
2 rows inserted.
```

B. DELETE operation

Definition: DELETE removes rows that match a condition. Without a WHERE clause, all rows are deleted, so the condition should be checked carefully.

```
DELETE FROM Student
WHERE student_id = 203;
```

```
OUTPUT
1 row deleted.
```

Important distinction: DELETE removes selected rows; TRUNCATE removes all rows efficiently in many DBMSs (with DBMS-specific transaction and identity behaviour); DROP removes the table definition itself.

C. Insertion anomaly

An insertion anomaly occurs when a table's structure prevents a fact from being added without adding unrelated or incomplete information. Example: if department details are stored only in Student(student_id, student_name, dept_id, dept_name), a new department cannot be recorded until at least one student joins it.

D. Deletion anomaly

A deletion anomaly occurs when deleting one fact unintentionally removes another fact that should be retained. Example: if the only student in a department is deleted from the combined Student table, the department's name and details may also disappear.

Solution – normalisation: separate entities into related tables and connect them with keys. Store department facts in Department and student facts in Student. This permits adding a department independently and deleting a student without deleting department information.

```
CREATE TABLE Department (
  dept_id INT PRIMARY KEY,
  dept_name VARCHAR(60) NOT NULL
);
```

```
CREATE TABLE Student (
  student_id INT PRIMARY KEY,
  student_name VARCHAR(80) NOT NULL,
  dept_id INT,
  FOREIGN KEY (dept_id) REFERENCES Department(dept_id)
);
```

Safety note: use transactions for related changes and verify the target rows before deleting. Foreign-key actions such as RESTRICT, CASCADE, and SET NULL should be chosen deliberately.

7. SQL INJECTION

SQL injection is a security vulnerability in which untrusted input changes the meaning of a SQL statement. It can expose, modify, or delete data and may sometimes lead to broader compromise of the application or database server.

Definition: SQL injection occurs when an application builds SQL commands by concatenating untrusted input into query text, allowing input to be interpreted as SQL syntax rather than as data.

How it happens:

- An application accepts input such as a username, search term, or account number.
- The application concatenates that input into a SQL string.
- A malicious input changes the intended condition or adds another SQL operation.
- The database executes the altered statement using the application's database privileges.

Illustrative unsafe pattern (do not use):

```
-- Pseudocode: unsafe string concatenation
sql = "SELECT * FROM Users WHERE username = '"
      + user_input + "'";
```

If input is treated as SQL syntax, an attacker may be able to bypass an intended filter. This example is conceptual; exact behaviour depends on the application and database.

Impact: unauthorised data reading, authentication bypass, unauthorised changes or deletion, exposure of personal information, and loss of trust or regulatory consequences.

Prevention techniques

- **Prepared statements / parameterised queries:** keep SQL code separate from user-supplied values.
- **Input validation:** validate type, length, format, and allowed values; validation complements but does not replace parameterisation.
- **Least privilege:** the application account should have only the permissions required for its tasks.
- **Safe stored procedures:** avoid unsafe dynamic SQL inside procedures.
- **Output and error handling:** do not reveal database errors, schema details, or secrets to end users.
- **Security testing:** use code review, static analysis, and authorised penetration testing.

Secure parameterised example (Python DB-API style):

```
# Values are bound separately from SQL text.
cursor.execute(
    "SELECT user_id, username FROM Users WHERE username = %s",
    (username_input,)
)
```

```
OUTPUT
The database treats username_input as a value, not as SQL code.
Placeholder syntax varies: SQLite commonly uses ?, while some drivers use %s or named parameters.
```

Real-world example: a shopping website searches for a customer's order by order number. A parameterised query ensures that the submitted order number is handled as data and cannot rewrite the SQL statement.

8. DATA WAREHOUSING VS. DATA MINING

Data warehousing and data mining are related but different concepts. A data warehouse stores integrated historical data for analysis; data mining discovers useful patterns and relationships within data.

A. Data Warehousing

Definition: a data warehouse is a centralised analytical data repository that integrates data from multiple sources and stores historical, structured data for reporting and decision support.

Important features:

- **Subject-oriented:** organised around subjects such as customers, sales, products, or finance.
- **Integrated:** combines data from different systems using consistent formats and definitions.
- **Time-variant:** retains historical snapshots so trends can be analysed over time.
- **Non-volatile:** data is typically loaded and queried rather than frequently changed by operational transactions.
- **ETL/ELT:** data is extracted from sources, transformed/cleaned, and loaded into analytical storage.

Real-world example: a retail chain combines sales from all branches, online orders, inventory, and customer data into a warehouse to compare monthly revenue and product performance.

Example analytical SQL:

```
SELECT product_id,
       SUM(quantity) AS units_sold,
       SUM(quantity * unit_price) AS revenue
FROM SalesFact
WHERE sale_date >= DATE '2026-01-01'
GROUP BY product_id
ORDER BY revenue DESC;
```

B. Data Mining

Definition: data mining is the process of discovering useful patterns, correlations, groups, trends, or anomalies in large datasets using statistical, machine-learning, and database techniques.

Important features:

- **Classification:** predicts a category, such as whether a transaction is likely to be fraudulent.
- **Clustering:** groups similar records without predefined labels, such as customer segments.
- **Association-rule learning:** finds items that frequently occur together, such as products commonly bought in one basket.
- **Regression and forecasting:** estimates numeric values or future trends.
- **Anomaly detection:** identifies unusual records that may indicate errors, fraud, or attacks.

Real-world example: a supermarket analyses purchase baskets and discovers that customers who buy pasta frequently also buy pasta sauce. The store may use this insight for product placement or promotions.

Simple association-rule illustration:

```
IF customer buys {bread, butter}
THEN customer often buys {jam}

Support    = fraction of all baskets containing all items
Confidence = fraction of {bread, butter} baskets also containing jam
Lift       = confidence / overall frequency of jam
```

Comparison

Aspect	Data warehouse	Data mining
Main purpose	Store and organise analytical data	Discover patterns and insights
Typical output	Reports, dashboards, historical views	Models, rules, clusters, predictions
Methods	ETL/ELT, dimensional modelling, OLAP	Statistics, machine learning, pattern discovery
Relationship	Often supplies clean data for mining	Can analyse warehouse data and other sources

In short: warehousing is primarily about collecting, integrating, and storing data for analysis; mining is about extracting patterns and knowledge from data. A warehouse does not automatically perform data mining, although the two commonly work together.

9. INTRUSION DETECTION IN DATABASES

Database intrusion detection monitors database activity to identify suspicious behaviour, policy violations, and possible attacks. It complements preventive controls such as authentication, authorisation, and encryption.

Definition: an intrusion detection system (IDS) analyses events or activity to identify signs of unauthorised access, misuse, or attack. A database activity monitoring or database IDS solution focuses on database users, queries, and events.

Types and approaches

- **Signature-based detection:** compares activity with known attack patterns. It is effective for known threats but may miss new or modified attacks.
- **Anomaly-based detection:** learns or defines normal behaviour and flags deviations, such as an account suddenly exporting thousands of records.
- **Host-based monitoring:** observes database server logs, processes, and file activity.
- **Network-based monitoring:** inspects database traffic moving across network connections.
- **Rule/policy-based monitoring:** alerts when activity violates rules, such as access outside approved hours or queries against restricted tables.

Important features: collect audit logs; correlate user, time, source, and query events; detect unusual access patterns; generate alerts; preserve evidence; support incident response; and integrate with a security information and event management (SIEM) system.

Real-world example: an employee account normally reads a few customer records each day but suddenly runs a query exporting the entire customer table at midnight. An anomaly detector or policy rule can alert the security team.

Illustrative monitoring query: SQL syntax and audit-view names differ by DBMS; this pseudocode represents the type of analysis a security team might perform over collected audit events.

```
SELECT db_user, COUNT(*) AS sensitive_reads
FROM AuditEvents
WHERE event_time >= CURRENT_TIMESTAMP - INTERVAL '1' HOUR
  AND object_name = 'Customer'
  AND action_name = 'SELECT'
GROUP BY db_user
HAVING COUNT(*) > 500;
```

```
OUTPUT
Flags users with more than 500 recorded SELECT events on Customer
during the last hour. A real deployment should account for legitimate
batch jobs, query aggregation, and the DBMS's audit-event format.
```

Limitations: false positives can overwhelm analysts; encrypted traffic may limit network inspection; poorly configured logging can miss events; and detection does not itself prevent every attack. Alerts must be investigated and connected to a response process.

10. SUMMARY AND CONCLUSION

A well-designed DBMS must do more than store records. It must represent queries precisely, maintain integrity, execute queries efficiently, protect sensitive information, and support reliable analysis.

- **Relational calculus** describes queries declaratively through tuple or domain variables.
- **Dependency preservation** helps enforce functional dependencies after decomposition.
- **Query optimisation** selects efficient execution plans using rewriting, statistics, indexes, and join algorithms.
- **Database security** protects confidentiality, integrity, and availability through layered controls.
- **DAC, MAC, and RBAC** provide different ways to decide who can access data.
- **INSERT and DELETE** modify rows, while normalisation helps prevent insertion and deletion anomalies.
- **SQL injection** is best prevented through parameterised queries, least privilege, and secure development.
- **Data warehousing** integrates historical data; **data mining** discovers patterns and predictions.
- **Intrusion detection** monitors activity to identify suspicious or unauthorised behaviour.

The topics are interconnected. Normalised schemas improve consistency, dependency preservation supports integrity, and query optimisation improves performance. Security controls protect the resulting data throughout its lifecycle, while warehouses and mining techniques help organisations turn reliable data into useful decisions.

REFERENCES

1. Silberschatz, A., Korth, H. F., and Sudarshan, S., *Database System Concepts*, McGraw-Hill.
2. Elmasri, R. and Navathe, S. B., *Fundamentals of Database Systems*, Pearson.
3. Ramakrishnan, R. and Gehrke, J., *Database Management Systems*, McGraw-Hill.
4. PostgreSQL Documentation, SQL Commands and Query Planning: https://www.postgresql.org/docs/
5. MySQL Reference Manual, SQL Statements and Security: https://dev.mysql.com/doc/refman/
6. OWASP Foundation, SQL Injection Prevention Cheat Sheet: https://cheatsheetseries.owasp.org/
7. NIST, Security and Privacy Controls and related cybersecurity publications: https://www.nist.gov/

— END OF ASSIGNMENT —